arXiv:2305.12050v2 [cs.SE] 16 Feb 2024

AI-assisted Code Authoring at Scale: Fine-tuning, deploying, and mixed methods evaluation

VIJAYARAGHAVAN MURALI, Meta Platforms Inc., USA
CHANDRA MADDILA, Meta Platforms Inc., USA
IMAD AHMAD, Meta Platforms Inc., USA
MICHAEL BOLIN, Meta Platforms Inc., USA
DANIEL CHENG, Meta Platforms Inc., USA
NEGAR GHORBANI, Meta Platforms Inc., USA
RENUKA FERNANDEZ, Meta Platforms Inc., UK
NACHIAPPAN NAGAPPAN, Meta Platforms Inc., USA
PETER C. RIGBY, Meta Platforms Inc., USA and Concordia University, Canada

Generative LLMs have been shown to effectively power AI-based code authoring tools that can suggest entire statements or blocks of code during code authoring. In this paper we present CodeCompose, an AI-assisted code authoring tool developed and deployed at Meta internally. CodeCompose is based on the InCoder LLM that merges generative capabilities with bi-directionality. We have scaled up CodeCompose to serve tens of thousands of developers at Meta, across 9 programming languages and several coding surfaces. We present our experience in making design decisions about the model and system architecture for CodeCompose that addresses these challenges.

To release a LLM model at this scale, we needed to first ensure that it is sufficiently accurate. In a random sample of 20K source code files, depending on the language, we are able to reproduce hidden lines between 40% and 58% of the time, an improvement of 1.4× and 4.1× over a model trained only on public data.

We gradually rolled CodeCompose out to developers. At the time of this writing, 16K developers have used it with 8% of their code coming directly from CodeCompose.

To triangulate our numerical findings, we conduct a thematic analysis on the feedback from 70 developers. We find that 91.5% of the feedback is positive, with the most common themes being discovering APIs, dealing with boilerplate code, and accelerating coding. Meta continues to integrate this feedback into CodeCompose.

CCS Concepts: • **Computing methodologies** → **Neural networks**; • **Software and its engineering**;

Additional Key Words and Phrases: AI, Developer productivity, Neural code completion, Program synthesis

ACM Reference Format:
Vijayaraghavan Murali, Chandra Maddila, Imad Ahmad, Michael Bolin, Daniel Cheng, Negar Ghorbani, Renuka Fernandez, Nachiappan Nagappan, and Peter C. Rigby. 2024. AI-assisted Code Authoring at Scale: Fine-tuning, deploying, and mixed methods evaluation. *Proc. ACM Softw. Eng.* 1, FSE, Article 48 (July 2024), 20 pages. https://doi.org/10.1145/3643774

Authors' addresses: Vijayaraghavan Murali, vijaymurali@meta.com, Meta Platforms Inc., USA; Chandra Maddila, cmaddila@meta.com, Meta Platforms Inc., USA; Imad Ahmad, imadahmad@meta.com, Meta Platforms Inc., USA; Michael Bolin, mbolin@meta.com, Meta Platforms Inc., USA; Daniel Cheng, danielcheng@meta.com, Meta Platforms Inc., USA; Negar Ghorbani, negargh@meta.com, Meta Platforms Inc., USA; Renuka Fernandez, rdfernandez@meta.com, Meta Platforms Inc., UK; Nachiappan Nagappan, nnachi@meta.com, Meta Platforms Inc., USA; Peter C. Rigby, Meta Platforms Inc., USA and Concordia University, Canada, pcr@meta.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2994-970X/2024/7-ART48
https://doi.org/10.1145/3643774

1 INTRODUCTION

Recently, large language models (LLMs)[12, 16, 36] have exhibited the ability to assimilate vast amounts of knowledge after being trained on the data in various corpora. This characteristic of LLMs has made them highly impactful in assisting developers in authoring code[2, 5, 6, 29]. Specifically, when they are trained to predict the next token in a code sequence, LLMs can become powerful coding assistants that can suggest entire statements or blocks of code during code authoring. Such a coding assistant trained on an organization's code repository can surface internal knowledge during code authoring when developers are most likely to seek that information.

At Meta, we built an AI-assisted code authoring system named CodeCompose to explore the application of LLM technology for code authoring. Meta is a large software company with a complex code base that covers a wide range of applications from social networking and virtual reality to software engineering infrastructure, such as continuous integration tooling and workplace coordination. Tens of thousands of developers work on billions of lines of code (LOCs) in a monolithic repository that hosts source code written in 9+ programming languages. One of the prominent activities in the Software Development Life Cycle (SDLC) at Meta is code authoring. A significant amount of contextual knowledge on the internal software development processes, and libraries, is embedded in the source code or is confined to a small set of developers. For example, a question like *"How can I upload a table to Hive in Hack?"* can be answered using that knowledge. The state of the source code and the developers associated with the source code keeps evolving constantly - internal libraries and tools get added and deprecated, while developers move across teams and change roles. At Meta's scale, keeping up with the knowledge required to accomplish coding tasks is challenging. Additionally, the dynamic environment at a large software company like Meta poses interesting challenges concerning knowledge discovery and achieving developer efficiency.

Over the past year, we (i) conducted R&D on the LLM (model) architecture, training data, and training objective, (ii) built an end-to-end coding assistant that offers code suggestions in various code authoring surfaces across the company, and (iii) gathered quantitative metrics and qualitative feedback to measure the impact of the CodeCompose system. CodeCompose has the following characteristics:

- Multi-lingual: CodeCompose is based on the InCoder LLM [18] and has been trained on 9+ programming languages at Meta. As such, it inherits the property of LLMs to be multi-lingual.
- Customized for the organization: CodeCompose is fine-tuned on Meta's internal code repository, and is thus able to handle languages such as Hack and Flow. CodeCompose is also deeply integrated with Meta's version of VSCode, as seen in Figure 1(a), and other editors.
- Natural language proficiency: CodeCompose can understand inline comments in natural language and generate code that adheres to the comment, as shown in Figure 1(b). It can also fluently generate comments, messages, and documentation.
- Bi-directional: CodeCompose can look beyond the cursor to suggest code at the current position. In Figure 1(c) it can be seen suggesting the docstring for a function from code that conventionally appears after the docstring.

In this paper we present how we built CodeCompose, discuss the unique challenges of our scale and how they influenced design decisions about CodeCompose, and finally, results from an extensive large-scale deployment including developer feedback on the impact of CodeCompose on how code is authored at Meta. Specifically, we provide evidence related to the following research questions:

RQ1. Model Evaluation: How well does CodeCompose generate one hidden line of code from existing code snippets?

Before designing and releasing a system to engineers, we test it on the existing code base. We randomly mask part of the code and allow the model to predict lines in the masked part to evaluate

```
elif isinstance(model_dir, dict):
    self.model_dirs = [model_dir["bucket"] + "/" + model_dir["dir"]]
if len(set(self.model_dirs)) < len(self.model_dirs):
    raise ValueError("Duplicate model directories detected.")
```

(a)

```
elif isinstance(model_dir, dict):
    self.model_dirs = [model_dir["bucket"] + "/" + model_dir["dir"]]
if len(set(self.model_dirs)) < len(self.model_dirs):
    # Kill the service!
    sys.exit("Duplicate model paths found. Please specify only one model path")
```

(b)

```
def is_service_running(self) -> int:
    """
    Returns 0 if service is running else non-zero.
    """
    if self.running:
        return 0
    else:
        return self.error_code
```

(c)

Fig. 1. CodeCompose (a) offers inline code suggestions in VSCode in a grey text appearing after the cursor when the user is typing code (Tab to accept), (b) changes its suggestion to adapt to a natural language comment, (c) suggests code or documentation based on code below the current position.

its accuracy. The code before and some of the code after the masked part is also fed to the model as the context, along with some additional metadata. We contrast the public InCoder model [18], against our fine-tuned model with both the Causal Masking (CM) and Language Causal Masking (LCM) objectives.

Result Summary: We find that LCM CodeCompose model exactly recreates the masked line between 40% and 58% of the time and has a high BLEU score between 56% and 73%, depending on the programming language. This represents an improvement over the Public model between 1.4× and 4.1×.

RQ2. Adoption: How many suggestions are accepted by engineers and what proportion of the code is written by CodeCompose?

Few studies of LLM for coding are able to go beyond backtests. There are some blog posts demonstrating the use by professional engineers. At Meta we are able to gradually rollout new features to engineers. We progressively roll CodeCompose out and monitor its impact through the number of accepted suggestions and the proportion of code that is written by CodeCompose.

Result Summary: In this paper, we make over 4.5M suggestions to 16K engineers. We see an acceptance rate of 22% which is comparable to those at Google and GitHub. We see that 8% of the changed code is written by CodeCompose, which is higher than the 3% reported by Google. CodeCompose is rolled out to 100% of engineers at Meta.

RQ3. Developer Feedback: How do developers perceive CodeCompose in their daily work?

We triangulate our numerical analysis, which show evidence of adoption, with feedback from developers and discussion of actual usages [15]. Using the feedback group for CodeCompose, we

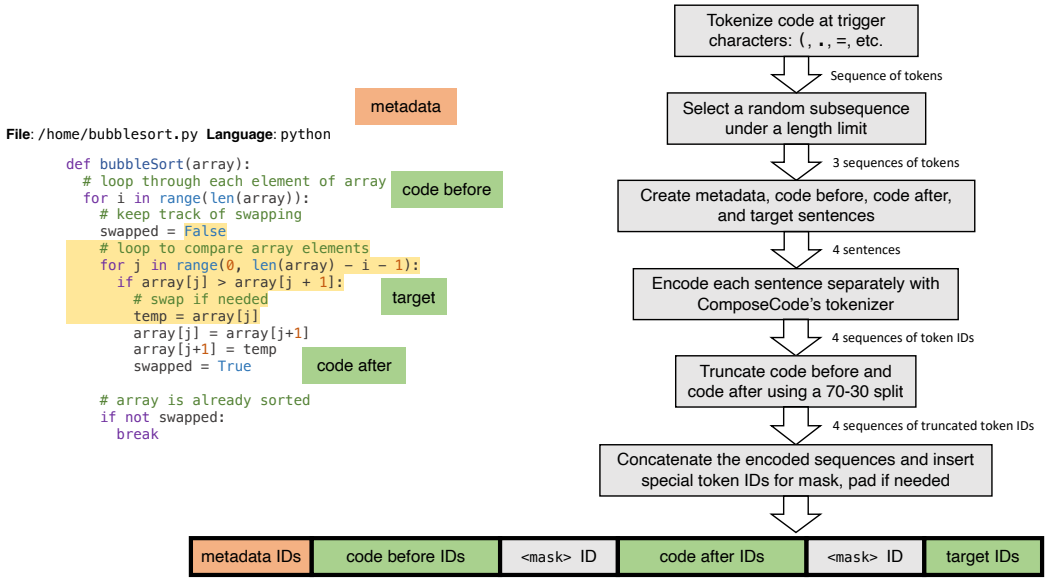

Fig. 2. Steps to construct an input to the model in LCM: (i) the code is tokenized at trigger characters where we expect the model to offer suggestions in production, (ii) a random subsequence of tokens is selected to be masked as the "target" to predict given the code before and the code after it, (iii) any additional metadata such as the filename is added to the front, (iv) all four strings (metadata, code before, target, code after) are encoded into tokens, (v) since the model's input length is limited, a 70-30 split is applied to the code before and code after if needed, (vi) all tokens are concatenated together into a single list of tokens with special tokens added to denote the masked target portion. The code on the left shows an example with the randomly selected target portion highlighted.

conduct a thematic analysis of the comments from 70 engineers. We provide direct quotes from engineers to contextualize and ground the themes we discovered. We also use these themes and feedback to iteratively improve CodeCompose.

Result Summary: The feedback was overwhelmingly positive, with only 8.5% of comments containing negative feedback and less than 1% of engineers turning off CodeCompose. The top two feedback comment themes related to discovering APIs and dealing with boilerplate (23%), and accelerating coding (20%). Meta continues to invest in CodeCompose efforts to iteratively improve the AI-assisted authoring experience.

The paper is organized as follows. Section 2 presents details about the CodeCompose model. Section 3 presents the system architecture. Section 4 presents results from our large-scale deployment at Meta. Section 5 discusses threats to validity. Section 6 discusses related work. Section 7 discusses the challenges for coding assistants at scale. Section 8 concludes the paper.

2 MODEL DEVELOPMENT AND EVALUATION METHODOLOGY

In this section, we provide details about the underlying LLM architecture for CodeCompose, and discuss considerations that affected our decision.

2.1 Model Architecture and Training Objective

LLMs largely fall into two main categories. The BERT [16] branch of models are trained with the Masked Language Model (MLM) objective that masks out certain parts of the input and trains the

model to predict them back. These models have a notion of bidirectionality, where predicting a token can take into account both preceding and following tokens. However, MLM is not easily suited for generative tasks. The GPT [12] branch of models are trained with the Causal Language Model (CLM) objective that provides the model with a sequence of tokens and trains it to predict the next token [12]. CLM makes the model more suited for auto-regressive generation tasks, however, it only takes into account the preceding tokens at a given point.

In industrial settings, developers often perform the task of editing where there is code before and after their cursor position. The code after the cursor contains highly relevant signals about the code being written. Thus, for a code generation system, we would ideally want both – a generative model with bi-directionality. For this purpose, we used the InCoder LLM [18] as our pre-trained model. InCoder is a generative model trained with the Causal Masking (CM) objective, where a sequence of tokens are masked out from a given input, appended to the end of the input, and the model is trained to generate the whole sentence left-to-right. Moreover, the tokenizer used by InCoder tokenizes multiple words into a single token, and as such allows efficient encoding of common code patterns such as `import numpy as np`. For more details, we refer the reader to the InCoder [18] paper.

To suit our use case, we make a few modifications to the CM objective and propose *Language Causal Masking (LCM)*:

- CM implements the masking after the text has been tokenized into token IDs, which limits the model during training to only seeing mask spans with edges at common tokenizer tokens. LCM lifts the masking step to the language level and avoids this, similar to the fill-in-the-middle (FIM) task [11]. Also, LCM only masks at certain trigger characters – that is, characters where the model will be queried during inference such as `(`, `.`, `=`, etc.
- We prefix certain metadata to the input in LCM, such as the programming language, full path to the file, and the kernel name for notebooks.
- Through model-level ablations, we found an optimal 70-30 split of the model's input length between code before and code after the cursor.
- Specialized for our use case, LCM has only one mask in any input.

A step-by-step overview of constructing an input in LCM is shown in Figure 2, along with an example code snippet. Once an input is constructed, during training, we maximize the log probability of the language-masked input:

$$\log \mathcal{P}(\texttt{[Metadata; Before; <mask>; After; <mask>; Target]})$$

where `Metadata`, `Before` and `After` are the tokens in the metadata, code before, and code after the cursor, respectively, `Target` is the code that was masked, and `<mask>` is a special token. During inference, we sample tokens in an auto-regressive manner from the distribution:

$$\mathcal{P}(\cdot \mid \texttt{[Metadata; Before; <mask>; After; <mask>]})$$

As we are suggesting lines of code, we stop the generation early once a newline token has been generated. Due to the real-time nature of our application and the inline suggestion user experience (UX), we only return one sequence of generated tokens.

2.2 Training data

InCoder has been trained on (i) a corpus of 159 GB of public code in 28 programming languages with permissive, non-copyleft, open-source licenses from GitHub and GitLab, and (ii) 57 GB of content from StackOverflow questions, answers, and comments. For training on our internal data, we collected data Meta's code repositories and notebooks, i.e., first-party data, applying several filters:

- Rather than crawling the entire repository, we used code that is modified through diffs (Meta's term for pull requests) checked in by developers as a way of staying close to our end

application (i.e., writing code in the IDE). This way we avoid training on code that may have been added a long time ago but is never modified.
- To keep the training data fresh, we only looked at diffs going back up to 2 years, and only kept the latest versions of files to avoid bugs that may have been patched.
- For each major target language, we exclude code that is not in production or deprecated.

After these filters, our first-party training data included in the order of tens of millions of files amounting to a few billion lines of code across 10+ languages.

We took InCoder-1.3B, the public model with 1.3 billion parameters[1], and fine-tuned it further on the above first-party data with our LCM objective. For fine-tuning the 1.3B model, we used a batch size of 20 per device (2.5 thousand effective) and a learning rate of 5e-4. Training for 4 epochs with sharded data parallelism took 4 days on a cluster of 128 A100 GPUs. We trained the model on all 9 languages at once. Following standard practice, the training data is randomized to help the model converge and to ensure that it does not overfit to a single language or dataset. We then deployed the model on a cluster of 150 A100 GPUs.

2.3 Model Evaluation Method and Measures

We describe our methodology to answer RQ1: How well does CodeCompose generate one hidden line of code from existing code snippets? Our methodology involves a series of evaluation experiments of CodeCompose that try to measure the accuracy of CodeCompose code generation in different languages and the impact of fine-tuning the model with first-party data, i.e., the Meta internal codebase. Our evaluation methodology follows a similar strategy to Figure 2, which we explain in detail in this section.

The experiments include four steps, namely test dataset collection, random masking, model evaluation, and metric measurements. Our test set is an unseen random sample of 20K source code files from Meta's source code repositories. Unseen samples come from a holdout set of files not used in training. The holdout set is separated by a filepath-based hash to ensure that there is no leakage of data between training and holdout. We collect files that belong to four major programming languages used in Meta: Hack, C++, Python, and Flow (JavaScript). Once we collect the test dataset, to replicate code completion scenarios in the IDE, we randomly mask a part of the code snippets ending with the end of a line, with a maximum length of 100 tokens, in each source code file. The length of the masked targets is on average 3% (or 39 characters) of the context passed to the model. We used heuristics to ensure that the masked part includes nontrivial spans of code, comments, etc. In this scenario, the prediction task is to predict the masked code snippets when the code before and after the masked part are passed to the model as the context. In addition to this context, we pass metadata such as file name, full path, and the programming language. Furthermore, to avoid over inflation of the model accuracy and represent a more realistic use case of CodeCompose code completion, we include extra masking of a random number of tokens after the target line to represent the not yet written part of the developers' code and feed the rest of the code snippet as the context to the model. The input context is consistent with the context at the model training, i.e., with the maximum size of 2k tokens and with the 70-30 split between the code before and after the masked target. While we use heuristics to make it as realistic as possible, we agree that the backtest may not reflect the actual coding patterns. To address this limitation, we employ multiple evaluation methodologies: backtesting (Section 4.1), online evaluation on the suggestions made in our production system (Section 4.2), and thematic analysis on user feedback (Section 4.3).

[1]At the time of acceptance of this paper, we have moved to using CodeLlama-7B [33] with 7 billion parameters as our foundation model. See [17] for more details.

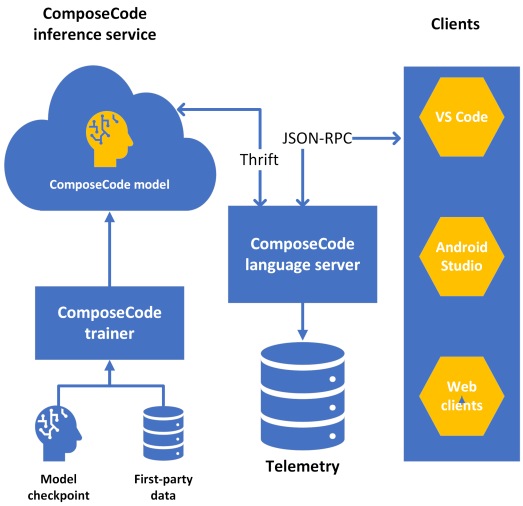

Fig. 3. System Architecture for CodeCompose. A pre-trained model checkpoint (InCoder) is trained by the trainer on first-party company data to create the fine-tuned CodeCompose model. This is served in a cluster of inference service machines with GPUs. When clients (VS Code, IDEs, web surfaces) require a code suggestion they send a JSON-RPC request to the CodeCompose language server protocol (LSP), which in turn communicates with the inference service through language-agnostic Thrift calls and sends the generated suggestion back. The LSP also logs telemetry which allows us to compute usage metrics, run experiments, and monitor for regressions. Clients, e.g., VS Code, then display the suggestion through their respective editors.

To evaluate the model, we run different versions of the model including the InCoder model as the public model and the fine-tuned model on the first-party data with the CM objective as well as the fine-tuned one with LCM objective. After running the different versions of the CodeCompose model, we measured two metrics and broke them down into the four main languages of Python, C++, Hack, and JavaScript to evaluate how well the model performs and the impact of fine-tuning with first-party data from Meta on the model accuracy. We use the measures of Exact Match (EM), as our most restricted metric, and BLEU score [30], which are used in similar studies [18, 27] and measured by comparing the reference of the masked part of the code and the predicted tokens by the model. The detailed results of this evaluation is described in Section 4.1.

3 SYSTEM DESIGN

The components of the CodeCompose system are shows in Figure 3. CodeCompose employs a typical client-server architecture in which the *server* is an inference tier that runs the model and the *client* is an editor that surfaces code suggestions, including VS Code and Android Studio. We encode the bulk of the client-side logic in a Language Server Protocol (LSP) [8] conformant language server that is reused across multiple editor integrations.

3.1 Server

At Meta, we have a tier of machines equipped with A100 GPUs, each with sufficient memory to host the fine-tuned CodeCompose model. Clients can make requests to this tier via Thrift [34]. The caller specifies the code before and after the cursor, as well as the file path, language, and metadata to use to process the request. The request handler is designed to be simple to utilize the most out of

the GPUs. It simply processes the request data into a string, tokenizes it, and invokes the model's auto-regressive generation function that runs on the GPU.

Since we are optimizing for latency rather than throughput, each request is processed as soon as it arrives and so there is no inference batching. Due to latency considerations, when the input length is less than 1500 tokens, we can afford to perform a beam search with a beam width of 3 and return the most likely suggestion. Otherwise, we perform greedy generation.

3.2 Language Server Protocol

To mediate requests between the client and server, we implemented a language server in Rust that we reuse across our various editor integrations. The LSP acts as an orchestrator and handles critical functionalities in the request-response workflow such as calling the inference APIs, displaying the received response as inline completions, logging the telemetry, etc.

While most LSPs are designed to support a wide array of traditional IDE functionality, such as autocomplete, jump-to-definition, the CodeCompose LSP supports only one meaningful request type: `textDocument/inlineCompletions` to help with displaying grey text inline completions.

The LSP also implements core request logic such as debouncing and caching. Debouncing waits for a 20ms pause before re-sending a request to avoid too many requests from being sent as the user is typing in the editor. Caching saves the response from the server with the context as the key, so that duplicate requests – which happen frequently as developers type and erase code – can be avoided. Moreover, requests that hit the LSP cache are served significantly faster than the service.

3.3 Clients

Clients are extensions that run locally on developers' machines and are responsible for ultimately displaying inline suggestions in the editor. For editors, such as VS Code that support LSP natively, we require relatively little glue code to create the CodeCompose extension. For editors such as Android Studio that does not have native LSP support, we built a small adapter to proxy requests to our LSP.

Furthermore, this architecture makes it straightforward to integrate CodeCompose with our in-house developer tools, such as the web-based variant of Jupyter notebooks. Because the notebook surface also supports LSP, it was easy to provide our data scientists and ML engineers with the same AI-powered code suggestions as developers working in VS Code. This ability to plug CodeCompose into any code editing surface internally is an advantage of owning the entire stack.

3.4 Telemetry

The LSP is responsible for recording all of the client-side telemetry, which ensures that metrics are recorded consistently across the various surfaces where CodeCompose is used. Because the LSP receives `textDocument/didChange` events, it can calculate most of the metrics of interest without further cooperation from the client. Nevertheless, the LSP does expect a client to send a custom `cc/received` notification upon receipt of the completions so that we can record the full round-trip time from the end user's perspective. Limiting the demands on the client facilitates integrating CodeCompose into more of our internal tools.

3.5 Evaluation Methodology for CodeCompose in Production

We used a mixed methods approach [15] to evaluate CodeCompose in production, collecting usage data for RQ2, and feedback comments for thematic analysis for RQ3. We follow the principles of Technical Action Research at every phase of the roll out [37]. This method of doing the rollout steadily in phases helped us measure the effects of CodeCompose in practice at every step (using the quantitative and qualitative feedback) and iterate on the product experience to improve the means to achieve intended outcomes before rolling out further. **Our rollout strategy** for CodeCompose

consists of gradual deployment in waves of languages: (i) only Python, (ii) Hack, Flow (Javascript), and C++, (iii) others. Within each wave, we rolled it out to increments of 25% of the developer population until we enable it for 100% of developers. The rollout was completed after four weeks in Spring of 2023.

A quantitative methodology is necessary to answer RQ2: How many suggestions are accepted by engineers and what proportion of the code is written by CodeCompose? We instrumented telemetry to track various events in the IDE such as displaying a suggestion inline, accepting or rejecting a suggestion, and the length of accepted suggestions. In total, our large-scale deployment resulted in CodeCompose making 4.5 million suggestions across 9 programming languages. 16 thousand distinct developers have seen at least one CodeCompose suggestion. We only count suggestions that were displayed for at least 750 milliseconds to ensure that developers were exposed to a suggestion and had a chance to see and comprehend it.

Our outcome measures are the acceptance rate of suggestions and the percentage of code typed using CodeCompose. These measures have been used in prior work, with, for example, Google reporting that 3% of the code typed by engineers was from their AI [5].

A thematic analysis is necessary to answer RQ3: How do developers perceive CodeCompose in their daily work?

We collected data by manually coding user posts in the CodeCompose feedback group. The first two authors coded responses and discussed them until a consensus was reached. In total, 70 engineers provided feedback. As starting codes, we used the standard positive, CSAT (Customer Satisfaction), or negative, DSAT (Customer Dissatisfaction). We then broke positive and negative feedback into subcodes. We then grouped codes and those that achieved saturation were abstracted into themes. For the major themes, we report the frequency of the themes. To ground our themes in the evidence, we provide representative quotations of both positive and negative feedback in Sections 4.3.1 and 4.3.2, respectively. This feedback also allowed us to reflect on CodeCompose and to identify the major factors affecting its perceived usefulness.

4 RESULTS

4.1 RQ1. Model Evaluation Results

How well does CodeCompose generate one hidden line of code from existing code snippets?

Before we can release CodeCompose into production, we need to ensure that the model is sufficiently accurate. We simulate an AI-authoring experience by selecting a random line from a file and providing the surrounding context to CodeCompose to determine how well it would have generated the hidden line. The training of the model and description of the metrics and evaluation methodology are in Section 2. We report the results for the number of exactly matching lines as well as the BLEU score for hidden lines in Table 1.

We compared the performance of various models such as the public checkpoint of the InCoder model and the fine tuned versions with different objectives. The Public model performs reasonably well. In Table 1, we see that the percentage of exact matches ranges from 14% for Hack to 23% for C++. The BLEU score is also reasonable ranging from 28% for Hack to 40% for C++.

Table 1 also shows the importance of fine-tuning the CodeCompose model on Meta source code so that it can learn Meta-specific code styles, formatting, and libraries. The LCM model always outperforms the CM model. The jump is dramatic especially in the case of LCM, with Hack seeing a 4.1 times improvement on the Exact Match and a 2.6 times improvement on the BLEU score over the Public model. The increase is still substantial for C++ with corresponding improvements of 1.7× and 1.4×, respectively.

Table 1. Outcome metrics across programming languages for the Public model and the fine-tuned models. Our LCM fine-tuned model substantially outperforms the Public model by between 1.4 times and 4.1 times.

Percentage of Exact Matches (improvement over Public Model by × times)								
Models	Python		Hack		Flow		C++	
Public	23.1%	–	14.2%	–	18.6%	–	23.4%	–
Fine-tuned (CM)	35.5%	1.5×	37.8%	2.7×	30.8%	1.7×	24.2%	1.0×
Fine-tuned (LCM)	48.8%	2.1×	57.7%	4.1×	52.2%	2.8×	40.0%	1.7×
BLEU Score (improvement over Public Model by × times)								
Models	Python		Hack		Flow		C++	
Public	37.9%	–	27.6%	–	33.8%	–	39.9%	–
Fine-tuned (CM)	50.0%	1.3×	52.6%	1.9×	45.8%	1.4×	40.3%	1.0×
Fine-tuned (LCM)	62.0%	1.6×	73.0%	2.6×	68.6%	2.0×	56.2%	1.4×

This experiment provides strong evidence that an internally fine tuned model outperforms an off-the-shelf model that is trained on external data only. The LCM model is very successful and is used in the subsequent sections where we roll out CodeCompose to developers.

> We find that LCM CodeCompose model exactly recreates the masked line between 40% and 58% of the time and has a high BLEU score between 56% and 73%, depending on the programming language. This represents an improvement over the Public model between 1.4× and 4.1×.

4.2 RQ2. Adoption Results

How many suggestions are accepted by engineers and what proportion of the code is written by CodeCompose?

We discuss the design of our CodeCompose system and its integration with VS Code and other editors used at Meta in Section 3. All changes at Meta must be gradually rolled out to users and we discussed our rollout strategy and adoption metrics in Section 3.5. Table 2 shows the adoption rates of CodeCompose.

Table 2. The acceptance rate and the proportion of code that was written by CodeCompose across programming languages

Language	# Suggestions shown	Acceptance rate	Code by CodeCompose	# Users
Python	1.9M	22%	8%	10.7K
Hack	1.3M	22%	10%	5.5K
C++	608.1K	20%	10%	2.5K
Flow (Javascript) [7]	583.2K	18%	7%	2.5K
Rust	74.2K	17%	9%	212
Objective C++	57K	18%	6%	429
Objective C	34K	18%	6%	299
C	23.5K	21%	12%	201
Typescript	8.9K	19%	10%	76
Total	4.5M	22%	8%	16K

CodeCompose made 4.5 million suggestions across 9 programming languages. 16 thousand distinct developers have seen at least one CodeCompose suggestion. As shown in Table 2, we observe a suggestion level acceptance rate of 22% across 9 programming languages for suggestions that were displayed for at least 750 milliseconds. Imposing a lower bound on the display time of the suggestions when calculating the acceptance rate helps in making sure developers who were exposed to a suggestion had a chance to see and comprehend it [5].

Our rollout strategy started with Python, which explains why there are more recommendations in that language. However, even typescript has over 8.9K recommendations and an acceptance rate of 19%, which is a substantial sample from a statistical perspective. CodeCompose has been rolled out to 100% of engineers at Meta.

We also log the length of each accepted suggestion, and the number of characters developers type when authoring code (excluding large copy-pastes, refactors, etc.). This allows us to calculate, at a character level, the percentage of the code typed by developers that came through accepting CodeCompose suggestions, which we computed to be 8%. CodeCompose is writing a substantial portion of the codebase, and in the next section we use developer feedback to understand what types of code are being accepted.

> We make over 4.5M suggestions to 16K engineers. We see an acceptance rate of 22% which is comparable to those at Google and GitHub. We see that 8% of the changed code is written by CodeCompose, which is higher than the 3% reported by Google. CodeCompose is rolled out to 100% of engineers at Meta.

4.3 RQ3. Developer Feedback Results

How do developers perceive CodeCompose in their daily work?

At Meta each internal tool has a feedback forum. The first two authors conducted a thematic analysis of the responses from 70 engineers (the methodology is described in Section 3.5). Similarly, the model does not differentiate between suggestion types such as code suggestions and documentation suggestion when making suggestions. We report the frequency of themes and suggestion types in Table 3. In subsequent sections, we provide grounded evidence of the themes through quotations and discuss both positive and negative feedback.

Table 3 lists the distribution of qualitative feedback responses. 91.5% of the CodeCompose users gave a favorable response while 8.5% of them gave an unfavorable response. Many respondents (15.7%) appreciated the fact that CodeCompose suggestions are accurate and CodeCompose was able to auto-complete the code they were going to type.

In addition to the accuracy of suggestions, 23% of the users said CodeCompose helped them discover new APIs, automating tedious tasks such as boilerplate code. Therefore, CodeCompose is not just helping with typing code faster (as demonstrated in Table 2) but also saving time and effort people spend searching for APIs and documentation.

CodeCompose is helping people move faster with with 20% of the users stating that they found CodeCompose to accelerating their coding activity. In addition to helping write code faster, CodeCompose is helping developers produce more in-code documentation and API documentation. This is an interesting side-effect and demonstrates the model's ability to generate text as an output from code in addition to code generation.

4.3.1 Evidence of Positive Experiences. After analyzing all the qualitative feedback, we identified the following main factors that make a developer inclined towards finding a system like CodeCompose useful.

Table 3. Frequency of themes coded from the feedback of 70 engineers. There are only six unfavorable responses, so we discuss each in Section 4.3.2

Theme	Description	Engineers
Accelerate coding	CodeCompose helped speed up my coding process	14 (20.0%)
Discovery	CodeCompose helped me discover APIs, write boilerplate code faster	16 (22.8%)
Documentation	CodeCompose helped me with generating in-code documentation, docstrings, etc.	5 (7.1%)
Suggestion accuracy	CodeCompose suggestions are accurate and not noisy	11 (15.7%)
Generally favorable	CodeCompose is generally great, and I will keep using it for my coding tasks	18 (25.7%)
Unfavorable	CodeCompose is not useful	6 (8.5%)
Total		70 (100%)

Favorable scenarios for CodeCompose: The scenarios for which CodeCompose was able to add the biggest value includes (but is not limited to), auto-completing lines of code, API discovery, boilerplate coding, suggesting standard libraries, generating in-code documentation, etc.

When it comes to developers, the developers who benefited the most are the ones who work on authoring code that involves writing boilerplate code and idioms, the ones who follow the typical coding patterns employed at Meta, and the ones that employ standard first-party (or third-party) libraries to accomplish their coding tasks. Additionally, CodeCompose is well received by the developers who tend to work on building pipelines and common infrastructure tasks that involve writing heavily templatized code. This can be corroborated by the distribution of responses listed in Table 3 and the representative quotes listed below.

Grounded evidence of positive experiences. We share quotations that exemplify the positive experiences of developers. A developer who got access to CodeCompose shared their first impressions:

> *"Today I needed to write a quick tool for* [task]. *I opened VS Code and started writing* [code]. *Suddenly this thing called CodeCompose popped up and wrote all the code for me! This was a super delightful experience, it really made my day, so thank you!"*

In this case, the developer was unaware of the existence of CodeCompose. This quote summarizes the delightful coding experience that the developers at Meta experienced when they pair-program with CodeCompose.

Several remarks are clear indicators of the usefulness of employing CodeCompose in code authoring workflows.

> *"I have been a big fan of* [CodeCompose] *ever since I got access. I do not think I have been in any coding session since then where I did not use it."*

> *"The suggestions* [from CodeCompose] *are quite relevant and do not show up too often – not necessarily "rarely", but not obnoxiously often either. It is a nice balance."*

> *"I was blown away that after writing the following* [code]*, CodeCompose successfully predicted the correct completion. This is amazing because I could not remember the name of the global variable or where it was in the code, but CodeCompose knew what it was."*

Many developers highlighted the fact that the predictions are relevant. Also, we received feedback about how nicely CodeCompose navigates the precision versus recall problem by not showing suggestions too often.

Several pieces of feedback highlighted CodeCompose's ability to help developers discover new APIs or ramp them up quickly on unfamiliar APIs.

> *"I wanted to post about a great experience I had in using CodeCompose in the* [internal] *codebase. I have been getting back into coding and was rusty on the different arguments that exist in the* [library] *operators. Some operators can have up to* [hundreds of] *arguments so this is a challenging problem. CodeCompose has helped me several times in a day by giving me the right argument that I cared about, whereas my previous workflow would have been to* [search] *for the operator and spend several minutes on each."*

> *"CodeCompose has been a game changer for working with APIs that I have not used before. Even if it is not exactly what I need, it often shows me approximately what I should pass or how I should call it. This is much faster than looking for other call sites to figure out how to use this* [API]*."*

We also observed interesting side effects such as increased in-code documentation when developers use CodeCompose to make code changes. As CodeCompose provides accurate descriptions of code changes and a template to start with, developers tend to accept the suggestions, make changes (as necessary), and push the documentation changes (along with the source code changes) in their diffs. This is reflected in some of the anecdotes listed below.

> *"I find CodeCompose particularly useful when writing docstrings. Without CodeCompose, I would not even imagine that I can write* [so many] *lines of docstring for my actual code."*

> *"I really like how CodeCompose highlights the value of naming and the quality of documentation, which used to be mainly for long-term benefit, but now good naming and* [documentation] *gives you an immediate effectiveness boost."*

4.3.2 Evidence of Negative Experiences. After analyzing the qualitative feedback, we found that CodeCompose is not helpful in scenarios where developers use specialized APIs and libraries. CodeCompose seems to be struggling with suggesting the correct code when the developers employ atypical coding patterns.

Some developers found the traditional, semantic, auto-complete functionality to be more useful in those cases. Developers also complained about the possibility of CodeCompose to hallucinate [19, 25] when recommending the uncommon API names, URLs, etc.

Developers also found the coexistence of CodeCompose with traditional auto-complete to be vital. Sometimes, when these two systems compete to show suggestions, it creates a disruptive experience for developers. Also, commonly used keyboard shortcuts such as "Tab" are overloaded to accept CodeCompose suggestions and traditional auto-complete suggestions. Solving these UX problems is vital to facilitating a smoother integration with existing semantic engines, which is addressed in our discussion Section 7.2.

Grounded evidence of negative experiences. A few developers disabled CodeCompose as they found CodeCompose to be noisy and intrusive. We list a few anecdotes below.

"CodeCompose is good, however, it seems to reduce my coding speed. The reason is that the traditional auto-complete was typically suggesting [real] *functions, but CodeCompose is suggesting hallucinations. These are not always correct and I would like a UX design where I could switch between traditional auto-complete and CodeCompose."*

"I have realized that [CodeCompose] *tends to struggle while working with function signatures. The suggestions it offers do not match the actual signature, although they make sense."*

"The [code editing] *experience is sometimes good and sometimes bad. Auto-complete struggles to work together with CodeCompose – they seem to be competing with each other."*

This highlights the importance of investing in UX research to deliver the best experience for AI-assisted code authoring while coexisting with traditional auto-complete systems. The feedback also talks about the problems faced by LLMs concerning hallucinations and grounding [19, 23]. We are actively exploring this area to make CodeCompose a productive experience for all the developers.

The feedback was overwhelmingly positive, with only 8.5% of comments containing negative feedback and less than 1% of engineers turning off CodeCompose. The top two feedback comments related to discovering API and dealing with boilerplate (23%), and accelerating coding (20%). Meta continues to invest in CodeCompose efforts to iteratively improve the AI-assisted authoring experience.

5 THREATS TO VALIDITY

5.1 Generalizability

CodeCompose is an entirely internal tool that has only been deployed at Meta. Therefore, there is a threat of drawing any general conclusions from the quantitative metrics or qualitative feedback presented. It is possible that the results might not hold elsewhere, and for this reason, we cannot assume a priori that other AI-based coding assistants built outside Meta will perform similarly. Similarly, the value-add of fine-tuning the LLM on Meta's internal code (Table 1) may also not translate externally. However, we see evidence of the need to fine-tune public models on internal data at Google [5]. Without this tuning on internal APIs and functions, the model would be unable to make reasonable suggestions for the internal codebase.

5.2 Construct Validity

There are many AI model evaluation metrics. We have selected the commonly used BLEU score and the percent of exact matches. While there are limitations with these metrics, exact match is a very conservative measure only succeeding when the code is perfectly generated.

There have been few deployments of AI reported in the literature, so we used standard measures employed at Meta, such as the level of adoption and the acceptance rate to quantify the efficacy of AI-authoring at scale. We triangulated these numerical results with a thematic analysis of feedback from 70 developers to ensure that the model's success resulted in a usable product.

5.3 Internal Validity

While we have taken significant measures to reduce bias in the results presented (e.g., by waiting several weeks after randomized rollout), we cannot guarantee that there is absolutely no bias. The time period chosen for measuring the metrics in Table 2 could have had external events beyond

our control that influenced developers' coding behaviors during that time. Some languages such as Typescript only has a small number of developers, who might have had an outsized influence on the metrics for those languages.

Qualitative feedback, in general, comes with a degree of subjectivity. While Meta encourages a culture of open feedback, developers might not have been comfortable sharing negative assessments with colleagues. We facilitated easy opt-out, but less than 1% had opted out, indicating that a majority of developers found CodeCompose to be a net positive experience.

The categorization of the qualitative feedback into different categories was done manually using a basic thematic analysis rather than a full grounded theory study. We tried to reduce subjectivity in this process by making two authors in the team categorize them and then reconcile the different labels. Nevertheless, a different set of people or more systematic methodology might have produced a different categorization in Table 3.

6 RELATED WORK

In this section, we review the related work and position our contribution in the literature. Neural program synthesis is a rapidly evolving area with a large body of work discussing the possibility of applying neural architectures, such as Bayesian networks, and Transformers, to solve program synthesis tasks [13, 14, 24, 31, 32, 38]. These models are typically evaluated against public data sets such as MBPP (974 Python programming tasks, designed to be solvable by entry-level programmers) [9] and Human Eval (164 original human-curated Python programming problems) [14]. The data set we used to evaluate the accuracy of code generation contains 20,000 data points sourced from real code bases, curated across 9 programming languages, at Meta.

There has been a limited deployment of AI-assisted code generation systems in large-scale industrial environments with tens of programming languages and thousands of developers using them daily [2, 4–6]. Availability of research in the public domain is even limited when it comes to evaluating the accuracy of neural program synthesis techniques on large scale industrial data sets and understanding the effects of such tools on software developers using mixed methods studies,

The closest in spirit to our work is the brief blog post from Google [5]. At Google, deploying a hybrid semantic ML code completion to 10k+ Google employees compared to a control group observed a 6% reduction in coding iteration time (time between builds and tests) when exposed to single-line ML completion. They reported that 3% of new code (measured in characters) was generated from accepting ML completion suggestions. The acceptance rate for single-line code completions is 25%. These results demonstrate the impact on developer productivity. In the OSS domain, GitHub CoPilot reported an acceptance rate of 30% for code completions [3].

Our more detailed findings are in line with those found by Google and GitHub with a 22% acceptance rate for 4.5 million single-line code completions across 9 programming languages. At the same time, our suggestions accounted for 8% of the code written by engineers at Meta compared to the 3% reported by Google.

There have been some empirical evaluations of GitHub's Copilot [2] in actual use for automatic code completion. In a grounded theory study of 20 developers using Copilot, Barke *et al.* [10] found that developers switched between coding acceleration and discovery use cases. Our engineers also reported these same themes as their most common positive feedback. Nguyen et al. [28] used 33 LeetCode questions to create queries for Copilot in four different programming languages. They found that Copilot's Java suggestions have the highest correctness score (57%) while JavaScript is the lowest (27%) [28]. While LeetCode examples are useful for algorithmic interview preparation, they differ dramatically from normal engineering work.

Code generation could also be achieved by prompting a general LLM such as ChatGPT [26]. Also, it has been found that different types of prompts can lead to different code generation performance.

We performed prompt engineering on the fine-tuned model (for e.g., determining the 70-30 split of the context fed to the model) to make it effective for code suggestions. We also show in Table 1 that public models underperform on Meta 's internal code base for code suggestion tasks compared to models fine-tuned on it.

A study with 15,000 synthetic data points curated from a C# code base from an industrial software development environment reported an accuracy of 30.7% for predicting single tokens, such as identifiers, when employing a modified Best Matching Neighbor approach (BMN#) [20]. Offline evaluations of Neural methods for auto-completion in a comparable industrial setting yielded accuracy of 41.9% on languages with heavily customized frameworks such as Hack [38] when tested on a couple of thousands of examples and when tested on open-source data sets (CodeSearchNet) [22] yielded an Exact Match score of 20.10% and BLEU score of 32.79% [27]. Our study of 20,000 data points curated, across 9 programming languages, from Meta's production code base shows that CodeCompose achieves the highest correctness score of 52% and BLEU score of 73% for Hack language and lowest correctness score of 40% and BLEU score of 56.2% for C++.

While Copilot is impressive, not all usage of has been positive. A qualitative study with 24 participants to understand how programmers use and perceive Copilot found that it did not necessarily improve the task completion time or success rate by a significant margin [35]. However, 19 of 24 participants (79%) answered that they preferred Copilot over Intellisense. A similar qualitative study of CodeCompose with a larger group of software engineers at Meta (70 participants) resulted in a 91.5% favorable feedback. Developers cited reasons such as accelerated coding or typing, helping in discovering new APIs and writing boilerplate code in unknown programming languages, helping with generating documentation for their code, accuracy of suggestions and its ability to predict what the developers are going to type as the main reasons.

7 DISCUSSION AND LESSONS LEARNED

We organize our discussion around the challenges and learnings in creating trust, making the user experience better, and employing metrics to evaluate AI coding assistants at scale.

7.1 Trust

Most of today's code generation LLMs consider code as a sequence of tokens, similar to natural language. That is one of the reasons for the observed acceleration in the model-building process – dealing with the complexities of parsing source code or developing semantic understanding is not required anymore. However, this has some side effects: most of the models cannot guarantee that the generated code compiles, is syntactically correct, or executes. For example, the model may make a "suggestion" by generating an API call that does not exist. Making developers trust and accept suggestions in this setting is a huge challenge. Other factors can also impact trust such as the source corpus used for training and its validity, biases in training data, security issues, vulnerabilities, constant data drift, etc.

The traditional precision versus recall problem in machine learning also plays into trust. If the model is optimized for recall and generates a lot of incorrect suggestions, developers will lose faith in the system as it becomes noisy. If the model is optimized for precision and generates only a few accurate suggestions, developers might eventually stop caring about the system as it becomes sparse. Also, it will be hard to justify the return on investment if the system is not generating enough suggestions. Studies[1] show that developers are fine with reworking a suggestion as long as the model provides a useful starting point or structure with relevant libraries and function calls. However, requiring a lot of rework might contribute to the reduction of trust and confidence in the model.

Learnings for CodeCompose. Building trust was an important part of our productization, as it is directly affected by model accuracy. First, we worked with language partner teams at Meta to identify obsolete patterns (e.g., "PHP-isms" in Hack) and not-in-production code (e.g., experiments) in the codebase and filter it out from our training data. In a similar vein, we only train on code that is being actively modified, that is, containing commits in the last two years, as opposed to all code in the repository.

Secondly, from developer feedback, we found out that contextual information adds significant value to the suggestion accuracy, such as the code *after* the cursor, the file being edited, or the kernel being used (in a notebook). We modified our model's training objective to take into account this contextual information (for more details see Section 2).

Finally, our rollout strategy allowed us to incrementally build trust in the product (details about our rollout strategy is explained in Section 3.5).

7.2 User Experience

Due to the generality of LLMs, code generation can be done at multiple levels of granularity: token completion, statement completion, completing multiple lines, or generating entire blocks. Depending on factors such as suggestion confidence, user context, and task context, we will need to generate suggestions at different granularities. For example, when developers are in their coding "flow", suggesting a large block of code and moving their existing code down – and moving it back up when they type over the suggestion – results in an extremely jarring experience. This is because developers would constantly need to switch between typing code and *reviewing* the suggested block of code, resulting in an unpleasant "code review on the fly" situation. Therefore, when to suggest, what to suggest, and how much to suggest is an essential part of the user experience.

Similarly, performance and latency are also an important consideration. Coding assistants operate in a real-time environment with strict requirements on latency. Developers do not want to wait for several seconds for the system to generate a single suggestion as they are typing it – the system has to match the developer's typing speed. Studies[21] have shown that sometimes users adjust their typing behaviors (and speed) to match autocomplete suggestions in IDEs. However, slowing oneself down should be compensated by passing the right suggestions and making it a "net positive" experience.

Learnings for CodeCompose. After several iterations, we found, through developer feedback, that offering suggestions one line at a time – that is, completing the current line of code where the cursor is – strikes a good balance between suggesting an adequate amount of code and avoiding the jarring effect. Furthermore, even within the current line, we do not show suggestions if there is any code to the right of the cursor, except certain tokens such as), }, and].

In terms of latency, developers were fine with suggestions appearing within 300ms - 500ms, and certainly not beyond 1s. We observed that the acceptance rate went down when the end-to-end latency went up. To bring down the end-to-end latency below this threshold, we employed a suite of optimizations such as caching and debouncing (for more details see Section 3). With this experience, developers who are looking for *multi-line suggestions* simply press Tab as many times as needed. This allows them to edit the currently suggested line before going to the next line, which makes the subsequent suggestions more accurate. They can also press a manual keyboard shortcut to request a multi-line suggestion, but we found that developers rarely use this workflow.

7.3 Metrics

Evaluating the usefulness of AI-generated suggestions is a major challenge[39]. This includes, but is not limited to, the complexities involved with the granularity of suggestions, developers' flexibility to rework the generated code, defining true positives, etc. A strict metric like Exact Match, BLEU

score or percentage of accepted suggestions as-is will underrepresent the actual benefits offered by these solutions. For example, while the acceptance rate is a good metric to measure product usage, a high acceptance rate does not necessarily mean developers are being more productive when using the tool – that has to be measured explicitly.

Furthermore, evaluation needs to go beyond just measuring product impact – it should help us understand the cases where the suggestions are inaccurate, or what the pain points are with respect to the user experience, to address those issues. Additional complexities including privacy, compliance, and proprietary data make it harder to track all the telemetry that is required to evaluate or monitor the product.

Learnings for CodeCompose. We developed a suite of metrics that can quantify our impact, some of which are standard definitions of internal metrics to measure the scale of deployment of tools at Meta. We track usage metrics such as the number of active users, user engagement and retention, number of suggestions offered, acceptance rate, and the percentage of code typed by developers, that comes from accepting CodeCompose's suggestions (for more details see Section 4).

8 CONCLUSION

In this paper, we introduced an AI-based coding assistant system named CodeCompose, discussed how we scaled it to 16 thousand developers across 9 programming languages at Meta, and presented metrics and feedback to understand the impact of the system on code authoring.

We made the following contributions:

We then presented the system architecture of CodeCompose. We built CodeCompose using three primary components: the model inference service, Language Server Protocol (LSP), and the client. When a developer types code, the service receives a request to perform inference and generates a sequence of tokens to auto-complete the statement(s). The LSP and the client help orchestrate the inference calls and implement the logic to display the suggestion.

We presented details about the underlying InCoder-based LLM that powers CodeCompose. We introduced a custom training objective, Language Causal Masking, that suits our application of suggesting individual lines of code. In doing so, we conducted an offline evaluation that showed a 1.4× and 4.1× improvement brought about by fine-tuning the LLM on the Meta's internal codebase.

We presented quantitative metrics that show the scale and impact of CodeCompose. 4.5 million suggestions were shown to developers resulting in 8% of code typed by users of CodeCompose coming from its suggestions.

We presented qualitative feedback from developers. We found that an overwhelming 91.5% of the feedback shared by developers was favorable towards CodeCompose, indicating that it adds significant value to their coding experience. Through a thematic analysis of the feedback, we found, for example, that CodeCompose helps developers discover unfamiliar APIs and write documentation. CodeCompose is now rolled out to all developers, and less than 1% of developers disabled it.

In the future, we plan to enable more features such as building dynamic context from other files to improve suggestion accuracy, incorporate feedback from user actions on suggestions to perform inference-time optimizations, more interaction modalities such as a conversational modality within the IDE, functionality to explain source code and provide code walk-throughs, etc. We are exploring opportunities to leverage semantic information to perform pre-processing and post-processing to improve suggestion accuracy by reducing hallucinations. Furthermore, we see opportunities to expand this technology, beyond code authoring, to help developers across the software development life cycle.

REFERENCES

[1] Github 2022. *How GitHub Copilot helps improve developer productivity*. Github. https://github.blog/2022-07-14-research-how-github-copilot-helps-improve-developer-productivity/

[2] Github Accessed 2021. *Github Copilot*. Github. https://github.com/features/copilot

[3] Accessed 2021. GitHub CoPilot's economic impact. https://github.blog/2023-06-27-the-economic-impact-of-the-ai-powered-developer-lifecycle-and-lessons-from-github-copilot/

[4] Microsoft Accessed 2021. *Microsoft Intellicode*. Microsoft. https://visualstudio.microsoft.com/services/intellicode

[5] Google Accessed 2021. *ML Enhanced Code Completion*. Google. https://ai.googleblog.com/2022/07/ml-enhanced-code-completion-improves.html

[6] Amazon Accessed 2023. *Amazon CodeWhisperer*. Amazon. https://aws.amazon.com/codewhisperer

[7] Accessed 2023. Flow Javascript. https://engineering.fb.com/2014/11/18/web/flow-a-new-static-type-checker-for-javascript

[8] Microsoft Accessed 2023. *Language Server Protocol (LSP)*. Microsoft. https://github.com/microsoft/language-server-protocol

[9] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732 [cs.PL]

[10] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. 7, OOPSLA1, Article 78 (apr 2023), 27 pages. https://doi.org/10.1145/3586030

[11] Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. 2022. Efficient Training of Language Models to Fill in the Middle. arXiv:2207.14255 [cs.CL]

[12] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs.CL]

[13] Marcel Bruch, Martin Monperrus, and Mira Mezini. 2009. Learning from examples to improve code completion systems. In *Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering (ESEC/FSE '09)*.

[14] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG]

[15] John W Creswell and Vicki L Plano Clark. 2017. *Designing and conducting mixed methods research*. Sage publications.

[16] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL]

[17] Omer Dunay, Daniel Cheng, Adam Tait, Parth Thakkar, Peter C Rigby, Andy Chiu, Imad Ahmad, Arun Ganesan, Chandra Maddila, Vijayaraghavan Murali, Ali Tayyebi, and Nachiappan Nagappan. 2024. Multi-line AI-assisted Code Authoring.

[18] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen tau Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. arXiv:2204.05999 [cs.SE]

[19] Nuno M. Guerreiro, Duarte Alves, Jonas Waldendorf, Barry Haddow, Alexandra Birch, Pierre Colombo, and André F. T. Martins. 2023. Hallucinations in Large Multilingual Translation Models. arXiv:2303.16104 [cs.CL]

[20] Vincent J. Hellendoorn, Sebastian Proksch, Harald C. Gall, and Alberto Bacchelli. 2019. When Code Completion Fails: A Case Study on Real-World Completions. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 960–970. https://doi.org/10.1109/ICSE.2019.00101

[21] Kajta Hofmann, Bhaskar Mitra, Filip Radlinski, and Milad Shokouhi. 2014. An Eye-Tracking Study of User Interactions with Query Auto Completion *(CIKM '14)*. Association for Computing Machinery, New York, NY, USA, 549–558. https://doi.org/10.1145/2661829.2661922

[22] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2020. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. arXiv:1909.09436 [cs.LG]

[23] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. *ACM Comput. Surv.* 55, 12, Article 248

(mar 2023), 38 pages. https://doi.org/10.1145/3571730

[24] Seohyun Kim, Jinman Zhao, Yuchi Tian, and Satish Chandra. 2021. Code Prediction by Feeding Trees to Transformers. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 150–162. https://doi.org/10.1109/ICSE43902.2021.00026

[25] Zihao Li. 2023. The Dark Side of ChatGPT: Legal and Ethical Challenges from Stochastic Parrots and Hallucination. arXiv:2304.14347 [cs.CY]

[26] Chao Liu, Xuanlin Bao, Hongyu Zhang, Neng Zhang, Haibo Hu, Xiaohong Zhang, and Meng Yan. 2023. Improving ChatGPT Prompt for Code Generation. arXiv:2305.08360 [cs.SE]

[27] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. arXiv:2102.04664 [cs.SE]

[28] Nhan Nguyen and Sarah Nadi. 2022. An empirical evaluation of GitHub copilot's code suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR '22)*.

[29] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. arXiv:2203.13474 [cs.LG]

[30] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics* (Philadelphia, Pennsylvania) *(ACL '02)*. Association for Computational Linguistics, USA, 311–318. https://doi.org/10.3115/1073083.1073135

[31] Sebastian Proksch, Johannes Lerch, and Mira Mezini. 2015. Intelligent Code Completion with Bayesian Networks. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 25, 1, Article 3 (12 2015).

[32] R. Robles and M. Lanza. 2008. How Program History Can Improve Code Completion. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. 317–326. https://doi.org/10.1109/ASE.2008.42

[33] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL]

[34] Mark Slee, Aditya Agarwal, and Marc Kwiatkowski. 2007. Thrift: Scalable cross-language services implementation. *Facebook white paper* 5, 8 (2007), 127.

[35] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems (CHI EA '22)*.

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. arXiv:1706.03762 [cs.CL]

[37] Roel Wieringa and Ayşe Moralı. 2012. Technical Action Research as a Validation Method in Information Systems Design Science. In *Design Science Research in Information Systems. Advances in Theory and Practice*, Ken Peffers, Marcus Rothenberger, and Bill Kuechler (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 220–238.

[38] Wen Zhou, Seohyun Kim, Vijayaraghavan Murali, and Gareth Ari Aye. 2022. Improving Code Autocompletion with Transfer Learning. In *2022 IEEE/ACM 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 161–162. https://doi.org/10.1145/3510457.3513061

[39] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity Assessment of Neural Code Completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming* (San Diego, CA, USA) *(MAPS 2022)*. Association for Computing Machinery, New York, NY, USA, 21–29. https://doi.org/10.1145/3520312.3534864

Received 2023-09-29; accepted 2024-01-23